Tutorial_8_Using_Plan_Mode

Tutorial 8: Using Plan Mode for Complex Projects

Duration: 45 minutes **Level**: Intermediate-Advanced

Overview

Master Plan Mode for structured development: - Two-phase planning - Option generation - Step-by-step execution - Approval workflow

Step 1: Define Complex Project

```
helixcode plan "Build a real-time chat application with:
- WebSocket server (Go)
- Redis for pub/sub
- PostgreSQL for message history
- React frontend
- User authentication
- Message encryption
- File attachments
- Typing indicators
- Read receipts
- Docker deployment"
```

Step 2: Review Generated Plan

```
📋 Plan Generated (ID: plan-chat-001)

## Real-Time Chat Application

### Complexity Analysis
- Estimated LOC: ~15,000
- Estimated Time: 8-10 hours (with AI)
- Difficulty: Advanced
- Technologies: Go, React, WebSocket, Redis, PostgreSQL

### Phase 1: Backend Foundation (2h)
1.1 Initialize Go project with gin and gorilla/websocket
1.2 Setup PostgreSQL schema (users, messages, rooms)
1.3 Implement connection pooling
1.4 Create Redis pub/sub manager
1.5 Add basic WebSocket handler

### Phase 2: Authentication & Security (1.5h)
```

```
2.1 Implement JWT authentication
2.2 Add end-to-end encryption (libsodium)
2.3 Create secure WebSocket upgrade
2.4 Implement rate limiting
2.5 Add CORS configuration

### Phase 3: Core Chat Features (2h)
3.1 Message sending/receiving
3.2 Room creation and management
3.3 Typing indicators
3.4 Read receipts
3.5 Message history pagination

### Phase 4: File Attachments (1h)
4.1 S3/MinIO integration
4.2 File upload API
4.3 Image thumbnails
4.4 Virus scanning integration

### Phase 5: Frontend (2h)
5.1 Create React app with TypeScript
5.2 Implement WebSocket client
5.3 Build chat UI components
5.4 Add message encryption/decryption
5.5 File upload interface

### Phase 6: Testing & Deployment (1.5h)
6.1 Unit tests for services
6.2 Integration tests for WebSocket
6.3 E2E tests with Playwright
6.4 Docker Compose configuration
6.5 Kubernetes manifests

Options generated: 5
```

Step 3: Explore Options

```
helixcode plan options plan-chat-001

# Option 1: Monolith (Recommended)
# - Single Go server
# - Embedded React build
# - Simple deployment
# ✓ Pros: Easy to develop, deploy, debug
# ✗ Cons: Harder to scale horizontally

# Option 2: Microservices
# - Auth service
# - Chat service
# - File service
# - API Gateway
```

```
# ✓ Pros: Scalable, independent deployment
# ✗ Cons: More complex, higher overhead

# Option 3: Serverless
# - AWS Lambda + API Gateway
# - DynamoDB
# - S3 for files
# ✓ Pros: Auto-scaling, pay-per-use
# ✗ Cons: WebSocket limitations, vendor lock-in

# Option 4: Firebase/Supabase
# - Use existing platform
# - Focus on frontend
# ✓ Pros: Fastest development
# ✗ Cons: Less control, ongoing costs

# Option 5: Hybrid (Microservices + Serverless)
# - Core chat: Microservices
# - File processing: Lambda
# ✓ Pros: Best of both worlds
# ✗ Cons: Most complex
```

Step 4: Select and Configure

```
# Select Option 1 (Monolith)
helixcode plan select plan-chat-001 --option 1

# Customize
helixcode plan configure plan-chat-001 \
  --set "database=postgresql" \
  --set "cache=redis" \
  --set "storage=s3" \
  --set "frontend=react-typescript"
```

Step 5: Execute with Approval

```
# Start execution
helixcode plan execute plan-chat-001

# HelixCode asks for approval at each phase

# Phase 1: Backend Foundation
# ✓ 1.1 Initialize Go project
# ✓ 1.2 Setup PostgreSQL schema
# ✓ 1.3 Implement connection pooling
# ✓ 1.4 Create Redis pub/sub manager
# ▮▮  1.5 Add basic WebSocket handler
#
```

```
# Preview changes? (y/n): y
# [Shows generated code]
#
# Apply changes? (y/n): y
# ✓ Phase 1 complete

# Continue? (y/n/skip): y
```

Step 6: Iterative Refinement

```
# During execution, request changes
helixcode plan refine plan-chat-001 \
  --phase 3.1 \
  --change "Add emoji support and reactions to messages"

# HelixCode updates plan and regenerates code
```

Step 7: Track Progress

```
helixcode plan status plan-chat-001

# Plan: Real-Time Chat Application
# Status: In Progress
# Progress: 45% (3/6 phases complete)
#
# ✓ Phase 1: Backend Foundation (complete)
# ✓ Phase 2: Authentication & Security (complete)
# ✓ Phase 3: Core Chat Features (complete)
# ⏳ Phase 4: File Attachments (in progress - 2/4 steps)
# ⏸  Phase 5: Frontend (pending)
# ⏸  Phase 6: Testing & Deployment (pending)
#
# Estimated time remaining: 3.5 hours
```

Step 8: Pause and Resume

```
# Pause execution
helixcode plan pause plan-chat-001

# Resume later (even on different machine)
helixcode plan resume plan-chat-001

# HelixCode restores context and continues
```

Step 9: Generate Documentation

```
# Auto-generate docs from plan
helixcode plan docs plan-chat-001

# Creates:
# - README.md
# - ARCHITECTURE.md
# - API.md
# - DEPLOYMENT.md
```

Step 10: Final Review

```
helixcode plan complete plan-chat-001

# Plan execution complete! 🎉
#
# Summary:
# - Files created: 89
# - Lines of code: 14,832
# - Tests written: 127
# - Coverage: 89%
# - Time taken: 9h 23m
# - Commits: 24
#
# Deliverables:
# ✓ Working chat application
# ✓ Complete test suite
# ✓ Docker deployment
# ✓ Comprehensive documentation
```

Advanced: Custom Plan Templates

```
# .helixcode/templates/microservice.yaml
name: "Microservice Template"
description: "Standard microservice with best practices"

phases:
  - name: "Project Setup"
    steps:
      - "Initialize Go module"
      - "Setup directory structure"
      - "Configure linting and formatting"

  - name: "Core Implementation"
    steps:
      - "Define domain models"
```

```
      - "Implement repository layer"
      - "Create service layer"
      - "Add HTTP handlers"

  - name: "Testing"
    steps:
      - "Unit tests"
      - "Integration tests"
      - "Contract tests"

  - name: "Deployment"
    steps:
      - "Dockerfile"
      - "Kubernetes manifests"
      - "CI/CD pipeline"

# Use custom template
helixcode plan create --template microservice \
  --name "inventory-service"
```

Results

- **Structured Development**: Clear phases and steps
- **Flexibility**: Multiple options, easy refinement
- **Traceability**: Complete execution history
- **Quality**: Automated testing and documentation

Best Practices

1. **Start with Plan Mode** for complex projects
2. **Review options** before committing to architecture
3. **Approve each phase** to maintain control
4. **Refine iteratively** based on results
5. **Save successful plans** as templates

All Tutorials Complete! 🎉

Return to Main User Manual

Sources verified

Sources verified 2026-05-29: https://go.dev/dl/ (go1.26.3 latest stable Go) ; https://www.postgresql.org/support/versioning/ (PostgreSQL 15+ in active support) ; https://redis.io/docs/latest/ (Redis 7+) ; project go.mod (inner go 1.26) + CLAUDE.md §3.1 (gin v1.11.0, gorilla/websocket v1.5.3, pgx/v5, go-redis/v9, PostgreSQL 15+, Redis 7+). The Plan-Mode example scaffolds a Go + gin + gorilla/websocket + PostgreSQL + Redis + Docker stack; all named technologies match the project's stack authority and no version pin in this tutorial was stale. NEGATIVE FINDING — `--set "database=postgresql"` / `--set "cache=redis"`

are generic stack selectors, not pinned versions; the orchestrator (Rule 4) supplies PostgreSQL 15+/Redis 7+ per §3.1.